# Actividades

# **Práctico-Experimentales** Nro. 004

## 1. Datos Generales

| | |
|---|---|
| **Asignatura** | Sistemas Distribuidos |
| **Ciclo** | 6to |
| **Unidad** | Uno |
| **Resultado de aprendizaje de la unidad** | R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 4 |
| **Título de la Práctica** | Mecanismos de Comunicación Inter-Proceso (IPC) y Red |
| **Nombre del Docente** | José O. Guamán Q. |
| **Nombre de integrantes** | Marco Castillo<br>VIviana Córdova<br>Isauro Rivera<br>Ariel Tandazo |
| **Fecha** | 13 de octubre de 2026<br>16 de octubre de 2026 |
| **Horario** | Martes 10h30 a 11h30 Viernes 07h30 a 09h30 |
| **Lugar** | Aula |
| **Tiempo planificado en el Sílabo** | 1 horas |

## 2. Objetivo(s) de la Práctica:

Implementar un sistema de asignación de pedidos que integre comunicación entre procesos (colas IPC) y comunicación en red (sockets TCP), y utilizar un agente de IA local (Ollama) provisto por el docente para la asignación de tareas, evaluando su comportamiento frente a la carga y la distancia.

**Objetivos específicos**

- Implementar comunicación entre procesos usando colas (IPC).
- Desarrollar comunicación entre computadoras mediante sockets TCP.
- Integrar un agente de decisión basado en IA usando Ollama.
- Simular un sistema real de asignación de pedidos (tipo delivery).
- Evaluar el comportamiento del sistema en función de la carga y la distancia.
- Analizar ventajas de usar IA frente a métodos tradicionales (reglas fijas).

## 3. Materiales y reactivos:

- Guia practica
- 2 o más computadoras (misma red WiFi/LAN)
- Router o red local

## 4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Ollama
- "baseURL": "http://10.20.202.199:3000/api","apiKey": "sk-c3c6fd2544f947d795c6e666ed21d431"

## 5. Procedimiento / Metodología

Fase 0: analizar y revisar los códigos

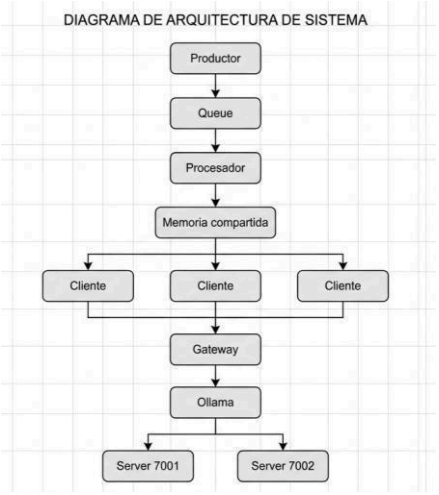



Fig 1: Arquitectura actual

| Script actual | Ubicación | Función |
|---|---|---|
| **app.py** | cliente_system/ | Es el punto de inicio actual del sistema. Inicia el sistema IPC y crea tres hilos clientes. |
| **cliente.py** | cliente_system/ | Lee datos de la memoria compartida y los envía mediante TCP al Gateway. |
| **main.py** | cliente_system/ipc_system/ | Crea la cola IPC, la memoria compartida y los procesos productor/procesador. |

| | | |
|---|---|---|
| **productor.py** | ipc_system/controls/ | Genera datos simulados de temperatura y vibración y los coloca en la cola. |
| **procesador.p y** | ipc_system/controls/ | Extrae datos de la cola, calcula una condición de alerta y guarda el resultado en memoria compartida. |
| **gateway.py** | gateway-system/ | Recibe datos de los clientes, consulta a Ollama y decide a qué servidor enviar los datos. |
| **server.py** | servidores-system/ | Actúa como nodo servidor. Recibe los datos mediante TCP y los muestra en pantalla. |

Fase 1: Preparación del entorno

Verificación de entorno: Python conexión con los servidores de la carrera modo API.

- Fase 2: Implementación del sistema

| Código actual | Nueva responsabilidad |
|---|---|
| **productor.py** | Generador de pedidos |
| **procesador.py** | Procesamiento de pedidos |
| **gateway.py** | Gateway de distribución |
| **Parte IA de gateway.py** | agente_ia.py |
| **server.py** | worker.py |
| **cliente.py** | Puede eliminarse o integrarse en gateway.py |
| **app.py** | Puede convertirse en main.py |
| **ipc_system/main.py** | Puede integrarse en main.py |

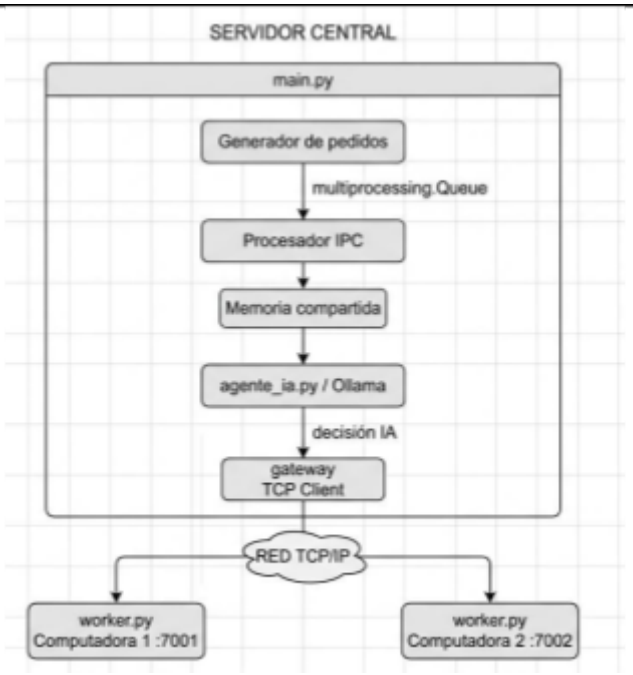



Fig 2: Arquitectura propueesta

1.IPC (cola de procesos)

Crear procesos que generen pedidos.

Usar multiprocessing.Queue para comunicación interna.

2. Agente inteligente

Implementar módulo agente_ia.py.

Conectar con Ollama vía API.

Diseñar prompt para toma de decisiones.

3. Comunicación en red

Implementar sockets TCP.

Configurar cliente (gateway) y servidor (worker).

```
Problems  Output  Debug Console  Terminal  Ports

viviana@viviana:~/proyectos/ape4_IPC$ cd practica4
viviana@viviana:~/proyectos/ape4_IPC/practica4$ python3 cliente_tcp 127.0.0.1 8080 "Hola servidor"
python3: can't open file '/home/viviana/proyectos/ape4_IPC/practica4/cliente_tcp': [Errno 2] No such file or directory
viviana@viviana:~/proyectos/ape4_IPC/practica4$ python3 cliente_tcp 10.20.203.144  "Hola servidor"
python3: can't open file '/home/viviana/proyectos/ape4_IPC/practica4/cliente_tcp': [Errno 2] No such file or directory
viviana@viviana:~/proyectos/ape4_IPC/practica4$ python3 cliente_tcp.py 10.20.203.144  "Hola servidor"
Uso: python cliente_tcp.py <ip_servidor> <puerto> <mensaje>
viviana@viviana:~/proyectos/ape4_IPC/practica4$ python3 cliente_tcp.py 10.20.202.199 3000  "Hola servidor"
[CLIENTE TCP] Conectado a 10.20.202.199:3000
[CLIENTE TCP] Enviando: Hola servidor
[CLIENTE TCP] Recibido: HTTP/1.1 400 Bad Request
date: Fri, 23 Oct 2026 14:26:46 GMT
server: uvicorn
content-type: text/plain; charset=utf-8
content-length: 30
connection: close

Invalid HTTP request received.
viviana@viviana:~/proyectos/ape4_IPC/practica4$
```

4. Nodo trabajador

Ejecutar worker.py en cada computadora remota. Simular procesamiento de pedidos.

5. Integración total

Ejecutar main.py en el servidor central.

Verificar flujo completo:

Generación → decisión IA → envío → ejecución

- Fase 3: Pruebas

Ejecutar múltiples pedidos simultáneos. Variar condiciones:

Distancia Carga de nodos

Observar decisiones del agente IA.

- Fase 4: Análisis
  Comparar:
  IA vs reglas fijas

Evaluar:

Tiempo de respuesta Distribución de carga

## 6. Resultados esperados:

- Correcta comunicación entre procesos (IPC).
- Envío de datos entre computadoras.
- Decisiones automáticas del agente IA.
- Distribución eficiente de pedidos

La implementación esta en este enlace: https://github.com/joseguaman1/distribuidos.git

## 7. Preguntas de Control:

- **¿Qué diferencia hay entre la comunicación IPC (cola de procesos) y la comunicación por sockets TCP?**

  Comunicación IPC: Este tipo de comunicación está diseñada exclusivamente para la sincronización y conexión de procesos que se ejecutan dentro de la misma máquina física o nodo. Para ello utiliza funcionalidad pertenecientes al kernel correspondiente al sistema operativo local tales como memoria compartida y otras infraestructuras internas, garantizando así tener un mayor rendimiento y un reducción de la latencia al no utilizar protocolos de red para su procesamiento.

  Comunicación por sockets TCP: Se emplea para conectar procesos en el cual no se encuentra en la misma computadora sino computadoras opuestas dentro de una red. Opera dentro de la capa de transporte, con el fin de entregar de forma ordenada y confiable los paquetes estableciendo un control de flujo, pero con cierto nivel de sobrecarga y latencia debido al procesamiento de paquetes con ciertos datos de seguridad adicionales.
- **¿Por qué el agente IA y el generador de pedidos pueden ejecutarse en computadoras distintas sin compartir memoria?**
  Principalmente dado al desacoplamiento de la arquitectura del sistema, al ser un sistema distribuido la mensajería dentro de la red se puede dar sin necesidad de compartir la memoria, esto es posible mediante la utilización de sockets TCP.
  Cada proceso opera en su propio entrono de ejecución, posteriormente realizan el intercambio estructurado de datos de forma transparente a través de la red.
- **¿Qué información debe viajar en el mensaje para que el agente decida correctamente? ¿Qué pasaría si faltara la carga del nodo?**
  Respecto a la información necesaria para el mensaje, para que el agente tome una decisión, este debe incluir los datos de la tarea o pedido, en donde incluya: Identificador, ubicación o destino, prioridad y requerimientos especiales. Adicionalmente el mensaje debe incluir el estado actual de los nodos que incluya en sí información de la métricas de carga de trabajo actual, distancia geográfica, latencia de red respecto a su destino y disponibilidad.

  En caso de que faltara esta carga del nodo, no podría evaluar el servidor respecto a su disponibilidad. Esto provocaría una distribución desequilibrada, lo que aumentaría su latencia y eficiencia en general del sistema.
- **¿Qué ocurre si un worker se desconecta mientras hay pedidos en la cola?**
  Obviamente producirá un fallo de comunicación o excepción en el socket TCP cuando se intente enviar un pedido o mantener algún tipo de enlace o comunicación, en caso de que cuente con cierto protocolos de tolerancia a fallos en caso de que no los mensajes podrían perderse o quedar bloqueados indefinidamente dentro del flujo de envío. También habría un

marcado de un nodo inactivo.

- **¿En qué casos un agente de IA aporta valor frente a reglas fijas y en cuáles no?**
  La IA aporta valor: En escenarios complejos y dinámicos donde intervienen múltiples variables simultáneas difíciles de modelar con fórmulas estáticas (por ejemplo, predicción de tráfico en tiempo real, priorización adaptativa basada en patrones históricos de demanda, fluctuaciones impredecibles del entorno y optimización multiobjetivo).

  Casos donde la IA NO aporta valor: En procesos deterministas, lineales y estrictos donde la lógica es simple y transparente (por ejemplo, un balanceo de carga clásico por Round-Robin o reglas fijas del tipo "si el nodo A tiene menos de 5 tareas, asígnale el pedido"). Tampoco es adecuado cuando se exige una latencia ultra-baja y predictibilidad absoluta, ya que consultar a un modelo de lenguaje requiere mayor costo computacional y tiempo de procesamiento que una evaluación algorítmica directa.

## 8. Declaración del uso de IA

Durante el desarrollo de la presente práctica y la posterior redacción de este informe técnico, se emplearon herramientas de Inteligencia Artificial Generativa bajo estrictos criterios de integridad académica y responsabilidad autoral. El uso de la IA se limitó exclusivamente a las siguientes funciones de apoyo metodológico:

- Estructuración y Guía Conceptual: Asistencia en la organización lógica de los apartados teóricos y técnicos para asegurar una secuencia analítica coherente con los objetivos de la asignatura de Sistemas Distribuidos.
- Consulta y Clarificación Técnica: Funciones de diccionario técnico para profundizar en la comprensión de conceptos asociados a la practica.
- Optimización de Redacción Académica: Apoyo en la revisión de estilo, corrección de redundancias y mejora del vocabulario técnico formal en los borradores redactados por el autor.

**Promts Utilizados**

**Prompt 1:** *"Necesito estructurar mi informe de práctica. Por favor, proponme un esquema de subtemas (con viñetas o títulos) que abarque: el concepto técnico, las diferencias principales, y un ejemplo aplicado al caso de estudio de distribución de pedidos. Ayúdame a mantener un flujo lógico antes de empezar a redactar."*

**Prompt 2:** *"Actúa como un glosario técnico. Necesito que me expliques de forma clara, concisa y con una analogía práctica el concepto de(Concepto de Ejemplo ). Incluye también por qué es importante dentro de la presente práctica y que aporte informativo dá."*

**Prompt 3:** *"Tengo el siguiente párrafo redactado por mí para la conclusión de mi práctica, pero siento que le falta formalidad técnica. Por favor, revísalo y mejóralo manteniendo mi idea central, pero utilizando un vocabulario más preciso y académico de ingeniería:...* **"***

## 9. Conclusiones

- A partir del análisis de la arquitectura base y su posterior evolución hacia el modelo distribuido, se concluye que la transición de scripts locales (productor.py, procesador.py) a nodos especializados (worker.py, gateway.py y el módulo modularizado agente_ia.py) logra un desacoplamiento óptimo; permitiendo combinar con éxito la comunicación local de alto rendimiento mediante colas IPC (multiprocessing.Queue) con la conectividad remota a través de sockets TCP.
- Mediante la preparación del entorno de red y la ejecución experimental de

múltiples pedidos simultáneos variando activamente la distancia y la saturación de los nodos (*workers*), se comprobó la estabilidad operativa del sistema y la capacidad del agente inteligente para procesar métricas en tiempo real y emitir decisiones de enrutamiento coherentes en escenarios de alta concurrencia.

## 10. Evaluación

NO APLICA

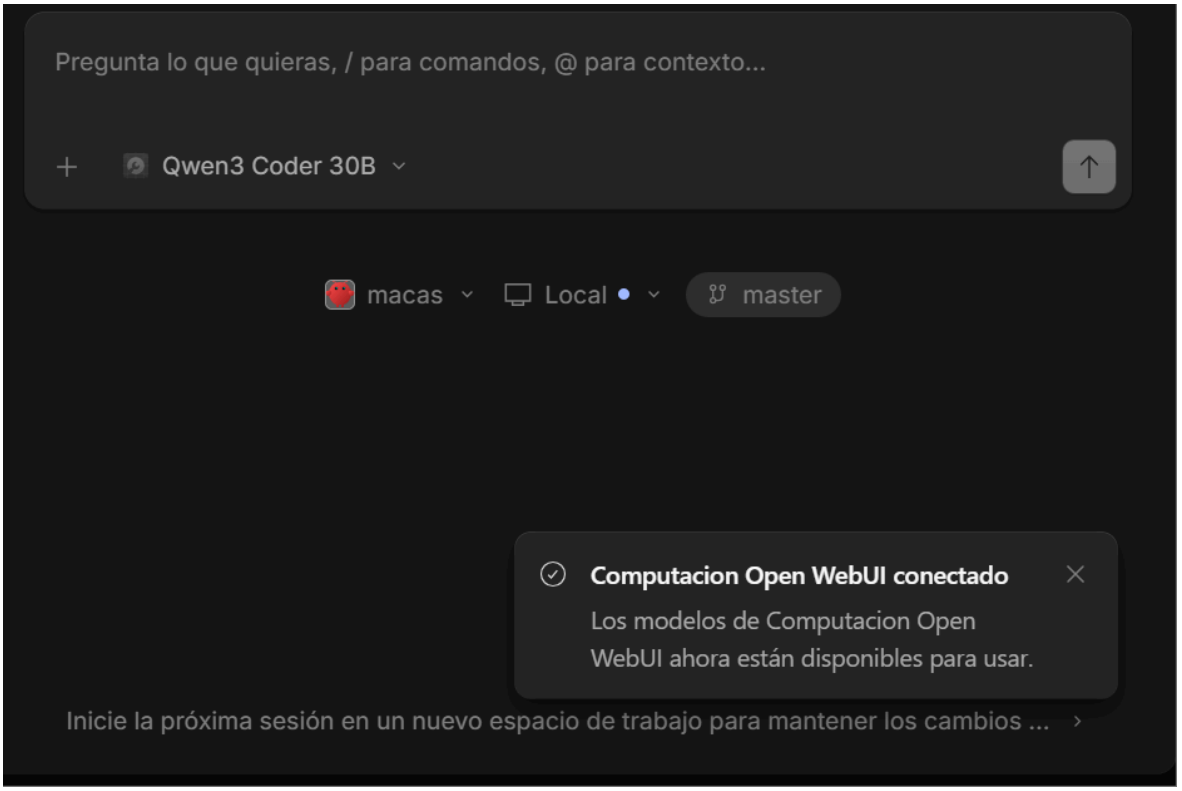
## 11. Bibliografía

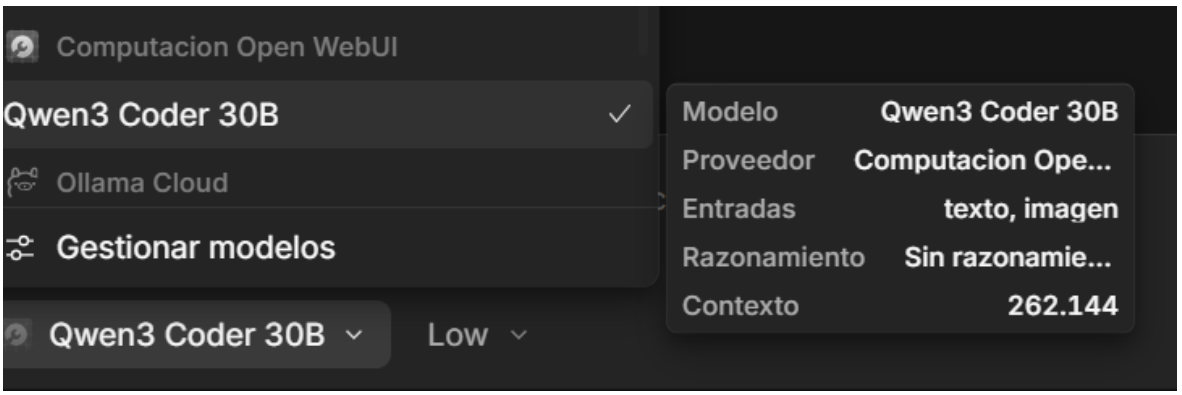
Semana 4. Guaman, Jose. 2026

## 12. Elaboración y Aprobación

| | | |
|---|---|---|
| **Elaborado por** | José Guamán<br>**Docente** | Validar únicamente en FirmaEC<br>electrónicamente por:<br>JOSE OSWALDO GUAMAN QUINCHE |
| **Ejecutado Por** | Rene Rivera<br>**Técnico Docente** | Validar únicamente en FirmaEC<br>electrónicamente por:<br>RAMIRO RENE RIVERA GUAMAN |
| **Aprobado por** | Edison L Coronel Romero<br>**Director de Carrera** | Firmado electrónicamente por: EDISON COMANDARI CORONEL ROMER<br>Validar únicamente con FirmaEC |

**Evidencias de conexión**






**1ER solicitud,.**

